

Engineer Head Interview – 50 Model Answers (Plain English)

1. Production issue ownership: I faced a login issue where users were logged out randomly. I traced it from UI to API to DB and found refresh token race conditions. Fixed it by making token rotation safe and added logs to prevent future issues.
2. Isolating bugs: I follow the data flow. Wherever data first becomes wrong, that layer is the problem.
3. Hardest bug: Background job running twice under load due to locking issue. Fixed with proper locks and idempotent logic.
4. Hotfix vs release: Hotfix for login, payment, data corruption. UI issues can wait.
5. Health metrics: Error rate, latency, retries, user complaints.
6. Crash reduction: Compared error logs before and after fixes under same traffic.
7. Never silent errors: Auth, payment, DB writes.
8. Error boundaries: Prevent full app crash; isolate broken UI.
9. Improvement: Better monitoring earlier.
10. Trade-off: Used Redis for speed, accepted invalidation complexity.
11. Event loop: Node handles tasks like a waiter giving long work to kitchen.
12. Long jobs: Split tasks, timeouts, progress tracking.
13. Duplicate jobs: Unique IDs, locks, idempotent logic.
14. Crash mid-job: Queue retries after restart.
15. Retry & backoff: Retry slowly to avoid overload.
16. Queue vs pub/sub: Queue = one consumer, pub/sub = many.
17. Memory leaks: Heap snapshots, monitor memory growth.
18. Avoid jobs: For real-time responses.
19. Too many workers: DB overload, CPU exhaustion.
20. Idempotent jobs: Same job twice gives same result.
21. MongoDB: Flexible schema, fast iteration.
22. SQL: Transactions, strong relations.
23. Slow Mongo: Missing indexes, large scans.
24. Index: Like book index.
25. Index downside: Slower writes.
26. Query optimization: Compound index reduced latency.
27. Redis cache vs session: Cache = speed, session = user state.
28. Invalidation: TTL or clear on update.
29. Redis down: Fallback to DB.
30. Stale data: TTL and invalidation.
31. Tokens: Access short, refresh long.
32. Refresh storage: HTTP-only cookies.
33. Long JWT risk: Token theft.
34. bcrypt: Slow hashing protects passwords.
35. Token theft: Rotate and logout.
36. RBAC: Backend enforcement.
37. API abuse: Rate limiting.
38. Validation: Reject bad input early.
39. Common mistake: Trusting frontend.
40. Auditing: Who, what, when logs.
41. Scale: DB bottleneck first.
42. First break: DB or network.
43. Socket scaling: Redis adapter.
44. Adapter role: Sync events.
45. Without adapter: Missed events.
46. Ordering: Sequence IDs.
47. Graceful shutdown: Finish requests then exit.
48. Monitoring: Errors, latency, memory.
49. Notifications: Event → queue → worker → retry.
50. Latency spike: Check logs, DB, network.